
bp_text

Release 0.0.1

Ruben Philipp

Apr 23, 2025

CONTENTS:

1	Usage	3
1.1	Installation	3
2	API	5
2.1	Modules	5
3	Examples	15
	Python Module Index	17
	Index	19

This is the documentation of `bp_text`, which is a Python library containing tools for algorithmic text generation.

At its core, `bp_text` is capable of processing various text files contained in a database (as of now in form of a BibTeX file), analyzing them (e.g. by tokenizing the PDF-text) and generating new material from the source text(s).

This project has been initially created for the project *the prospect of its own undoing* (working title) by Fabian Bentrup and Ruben Philipp.

Note

This project is still under active development. Thus, the API is likely to change. Additionally, not all functionality described in parts of this documentation is implemented.

1.1 Installation

The easiest way to install this package is by calling:

```
pip install bp_text  
# or with an absolute path  
pip install /path/to/bp-text/python/
```


This documentation is automatically generated from the sources.

2.1 Modules

<code>bp_text.utilities</code>	This module implements utility functions.
<code>bp_text.database</code>	This module implements database functionality.
<code>bp_text.page</code>	This module implements the page class.
<code>bp_text.language</code>	This module contains functionality dealing with languages (e.g. detection).
<code>bp_text.pdf</code>	This module implements functionality for PDF files.
<code>bp_text.txt</code>	This module implements functionality for TXT files.

2.1.1 bp_text.utilities

This module implements utility functions.

Created: 2025-03-28 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 16:39:52 Wed Apr 23 2025 CEST

Functions

<code>file_checksum(file_path[, algorithm, chunk_size])</code>	This function creates a checksum for a file.
<code>read_pickle(file_path)</code>	Read a serialized (pickled) file and return the data it holds.
<code>write_pickle(obj, file_path[, protocol])</code>	This function serializes (pickles) an object and stores it in a file given as <i>file_path</i> .

`bp_text.utilities.file_checksum(file_path, algorithm='sha256', chunk_size=8192)`

This function creates a checksum for a file.

Parameters

- **file_path** (*string*) – Path to the file to be checksummed.
- **algorithm** (*string*) – The hash algorithm (e.g. “sha256”, “md5”, “sha1”). Default = “sha256”
- **chunk_size** (*integer*) – The chunk size for checksum creation in Bytes. Default = 8192

`bp_text.utilities.read_pickle(file_path)`

Read a serialized (pickled) file and return the data it holds.

Parameters

file_path (*string*) – The path to the serialized data.

`bp_text.utilities.write_pickle(obj, file_path, protocol=5)`

This function serializes (pickles) an object and stores it in a file given as *file_path*.

Parameters

- **file_path** (*string*) – The path where to store the pickle file.
- **protocol** (*integer*) – The pickle protocol. Default = *pickle.HIGHEST_PROTOCOL*

2.1.2 bp_text.database

This module implements database functionality. Its main purpose is to read from a BibTeX file (as database).

Created: 2025-03-23 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 17:32:55 Wed Apr 23 2025 CEST

Functions

<code>convert_latex_umlauts(text)</code>	Convert LaTeX umlauts to ASCII umlauts.
--	---

Classes

<code>BibTexDatabase(file_path[, split_keywords, ...])</code>	Implementation of a BibTeX database.
<code>Database()</code>	An abstract superclass for a database.

class `bp_text.database.BibTexDatabase(file_path: str, split_keywords=True, split_files=True)`

Bases: `Database`

Implementation of a BibTeX database. This class provides the user with various methods to interact with the stored data.

NB: This class is limited to read-only use. In order to modify the contents of the actual database it is recommended to use a specialized software (e.g. BibDesk).

The *data* attribute contains the parsed data.

Example:

```
database = database.BibTexDatabase("db.bib")
database.entries["adorno1960"].get("file").value
```

Parameters

- **file_path** (*string*) – The path to the .bib file.
- **split_keywords** (*boolean*) – When true, split all keywords in the *keywords* field into a list, assuming they are separated by a comma (“,”). Default = True
- **split_files** (*boolean*) – When true, split all files in the *file* field into a list, assuming they are separated by a semicolon (“;”). Default = True

__init__(*file_path*: str, *split_keywords*=True, *split_files*=True)

property data

The parsed data.

property entries

The database entries as a *dict*.

find_entries(*field*: str, *search*: str)

Find entries matching the search string in the given field.

Parameters

- **field** (*string*) – The field name (e.g. “keywords”).
- **search** (*string*) – The search string.

Returns

A list with items of <class ‘*bibtexparser.model.Entry*’>

Return type

list

get_entry_by_key(*key*)

Get a specific entry by citation-key in the db.

Parameters

key (*string*) – The citation key (e.g. “@adorno1960”) to look for.

get_nth_entry(*n*)

Get the nth entry in the database.

Parameters

n (*integer*) – Index (zero-based) of the entry in the database.

load(*file_path*: str, *split_keywords*=True, *split_files*=True)

Load and parse a BibTeX file.

Parameters

- **file_path** (*string*) – The path to the BibTeX file.
- **split_keywords** (*boolean*) – When true, split all keywords in the *keywords* field into a list, assuming they are separated by a comma (“,”). Default = True
- **split_files** (*boolean*) – When true, split all files in the *file* field into a list, assuming they are separated by a semicolon (“;”). Default = True

split_fields_by(*field*: str, *separator*=’,’)

Splits the data/value of all entries in the database (destructively) of the given field (e.g. “keywords”) by a given separator.

Parameters

- **field** (*string*) – The field name (e.g. “keywords”).
- **separator** (*string*) – The separating character. Default = “,”

class bp_text.database.Database

Bases: ABC

An abstract superclass for a database.

abstractmethod load(*file_path: str*)

Load a database from a file.

Parameters

file_path (*string*) – The path to the database file.

bp_text.database.convert_latex_umlauts(*text*)

Convert LaTeX umlauts to ASCII umlauts.

Parameters

text (*string*) – Text to convert.

2.1.3 bp_text.page

This module implements the page class.

Created: 2025-03-28 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 16:58:40 Wed Apr 23 2025 CEST

Classes

Page([*page_num*, *page_label*, *data*, *text*, *lang*])

Abstract base class for a page.

class bp_text.page.Page(*page_num=None, page_label=None, data=None, text="", lang=""*)

Bases: ABC

Abstract base class for a page.

Parameters

- **page_num** (*int*) – The page number (zero-based) of the page in the related file.
- **page_label** (*string*) – The actual page label of the page. Its value and meaning differs from the *page_num* as it is related to the actual page numbering e.g. in a document. Thus, it could also be a roman numeral or be counted from a starting index different from 0.
- **data** (*undefined*) – Holds page data.
- **text** (*string*) – Holds the actual text of the page.
- **lang** (*string*) – The language code of the primary language in the alpha3/ISO 639-2 form.

__init__(*page_num=None, page_label=None, data=None, text="", lang=""*)

count_words()

Counts the words in the text.

Returns

The number of words in the text.

Return type

integer

property data

detect_lang(*set_lang=True*)

Detect the primary language of text in the page.

Parameters

set_lang (*boolean*) – When true, automatically set the language attribute of the page. Default = True.

property lang

property page_label

property page_num

property text

update()

Updates the instance.

2.1.4 bp_text.language

This module contains functionality dealing with languages (e.g. detection).

Created: 2025-03-27 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 17:16:42 Wed Apr 23 2025 CEST

Module Attributes

<i>default_languages</i>	The default languages used by the detection algorithm.
--------------------------	--

Classes

<i>LanguageDetector</i> ([languages])	A language detector.
---------------------------------------	----------------------

```
class bp_text.language.LanguageDetector(languages=[Language.ENGLISH, Language.FRENCH,
                                             Language.GERMAN, Language.SPANISH])
```

Bases: object

A language detector.

Parameters

languages (list of *lingua.Language* objects.) – A list containing all languages (cf. *lingua.Language*) to consider. Default = *default_languages*

```
__init__(languages=[Language.ENGLISH, Language.FRENCH, Language.GERMAN,
                    Language.SPANISH])
```

property detector

This is the actual detector object to use for detection.

Example:

```
detector = language.LanguageDetector().detector
detector.detect_language_of("Hallo Welt")
```

property languages

update()

Update the instance.

`bp_text.language.default_languages = [Language.ENGLISH, Language.FRENCH, Language.GERMAN, Language.SPANISH]`

The default languages used by the detection algorithm.

2.1.5 bp_text.pdf

This module implements functionality for PDF files.

Created: 2025-03-27 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 17:37:49 Wed Apr 23 2025 CEST

Classes

<code>PdfFile(file[, auto_extract, use_ocr, ...])</code>	This is a class implementation of a PDF file.
<code>PdfPage([page_num, page_label, data, text, ...])</code>	This is a class implementation for a PDF page.

```
class bp_text.pdf.PdfFile(file: str, auto_extract=True, use_ocr=False, fallback_to_ocr=True, ocr_dpi=300,
                           ocr_default_lang='eng')
```

Bases: object

This is a class implementation of a PDF file. A PDF file object is related to an actual PDF file (e.g. retrieved from a database entry). Its methods facilitate e.g. the retrieval of data/text from the pages.

The *data* attribute holds the *PdfPage* objects.

Examples:

```
## load a PDF file and extract the content from its pages
pdfFile = pdf.PdfFile("bajohr2024a.pdf", auto_extract=True)

## get the primary language
print(pdfFile.lang)
## => "de"

## get the label from the second page (in this case a roman numeral)
pdfFile.data[1].page_label
## => "II"

## get the text from the third page
pdfFile.data[2].text
```

Parameters

- **file** (*string*) – The filepath.
- **auto_extract** (*boolean*) – Automatically extract the text from all pages in the file when instantiating the object? This also automatically creates *PdfPage* objects for each page. Default = True
- **use_ocr** – Use OCR by default for text extraction? Default = False
- **fallback_to_ocr** (*boolean*) – If text extraction without OCR yields little text, fallback to OCR? Default = True

- **ocr_dpi** (*integer*) – The DPI amount for OCR. Default = 300
- **ocr_default_lang** (*string*) – The default language for OCR. Default = “eng”

__init__ (*file: str, auto_extract=True, use_ocr=False, fallback_to_ocr=True, ocr_dpi=300, ocr_default_lang='eng'*)

property auto_extract

Do auto-extraction?

property data

The data.

extract_text()

Extract text from a PDF using direct extraction or OCR. Returns a list of PdfPage objects.

extract_text_with_ocr()

Extract text from a PDF using Tesseract OCR. Returns a list of PdfPage objects.

extract_text_without_ocr()

Extract text from a PDF using PyPDF2. Returns a list of PdfPage objects.

property file

The filepath to the PDF.

property file_checksum

Checksum (SHA256) of the PDF file (read-only).

get_page_label (*page_num*)

Returns the label (i.e. the page number according to the PDF number tree) of a pdf page by index (page_num, zero-based).

Parameters

- **page_num** (*integer*) – The page number (zero-based) the label should be retrieved from.

get_primary_lang()

Get the primary language of a PDF.

property lang

The language.

property reader

The *PyPDF2.PdfReader* object (read-only).

update()

Updates the instance.

class bp_text.pdf.PdfPage (*page_num=None, page_label=None, data=None, text="", lang="", file=None*)

Bases: [Page](#)

This is a class implementation for a PDF page. A PDF page holds is a reference to a page in a PDF document, usually related to a [bp_text.pdf.PdfFile](#) object.

The data attribute is also capable of holding a *PyPDF2.PageObject* (optional), while the text attribute can contain text retrieved from the file (e.g. via OCR or direct extraction – cf. [PdfPage.extract_text\(\)](#) or [PdfFile.extract_text\(\)](#)).

Parameters

- **page_num** (*int*) – The page number (zero-based) of the page in the related file.

- **page_label** (*string*) – The actual page label of the page in the PDF file. The actual PDF page number/label is defined in the PDF header and could differ from the *page_num* (e.g. by varying the start index being a roman instead of an arabic numeral).
- **data** (A *PyPDF2.PageObject*) – A *PyPDF2.PageObject*. Default = None
- **text** (*string*) – The text contained in the page (when retrieved).
- **lang** (*string*) – The language code of the primary language in the alpha3/ISO 639-2 form.
- **file** (A *PdfFile* object.) – An optional (back-)reference to a PdfFile object.

__init__(*page_num=None, page_label=None, data=None, text="", lang="", file=None*)

Constructor method.

count_words()

Counts the words in the text.

Returns

The number of words in the text.

Return type

integer

property data

Read/write property for the data of the object.

detect_lang(*set_lang=True*)

Detect the primary language of text in the page.

Parameters

set_lang (*boolean*) – When true, automatically set the language attribute of the page. Default = True.

extract_text(*update_text=True*)

Extract text from a PDF page using direct extraction. Returns the text as a string.

Parameters

update_text (*boolean*) – Update the text attribute with the extracted text?

Returns

The retrieved text.

Return type

string

property file

Read/write property.

property lang

property page_label

property page_num

property text

update()

Updates the instance.

2.1.6 bp_text.txt

This module implements functionality for TXT files.

Created: 2025-03-29 Author: Ruben Philipp <me@rubenphilipp.com>

\$\$ Last modified: 17:39:37 Wed Apr 23 2025 CEST

Classes

<i>TxtFile</i> (file[, lang, data])	Implementation of the text-file (txt) class.
-------------------------------------	--

class bp_text.txt.TxtFile(file: str, lang="", data=None)

Bases: object

Implementation of the text-file (txt) class.

Example:

```
## instantiate the text file object and read its contents
text = txt.TxtFile("something.txt")
## get the primary language
print(text.lang)
## => "en"
```

Parameters

- **file** (*string*) – The path to the text file.
- **lang** (*string*) – The language of the text file (e.g. “en”, “de” etc.).
- **data** (*string*) – The content of the text file. This will be automatically set by reading the data from the file(-path).

__init__(file: str, lang="", data=None)

property data

Getter/setter for the data (i.e. the txtfile content).

property file

Getter/setter for the file-path.

property file_checksum

get_primary_lang()

Detect the primary language of the text in *data* and set the *lang* attribute accordingly.

property lang

Getter/setter for the language.

update()

Updates the instance.

EXAMPLES

TODO...

PYTHON MODULE INDEX

b

- `bp_text.database`, 6
- `bp_text.language`, 9
- `bp_text.page`, 8
- `bp_text.pdf`, 10
- `bp_text.txt`, 13
- `bp_text.utilities`, 5

Symbols

`__init__()` (*bp_text.database.BibTexDatabase* method), 6
`__init__()` (*bp_text.language.LanguageDetector* method), 9
`__init__()` (*bp_text.page.Page* method), 8
`__init__()` (*bp_text.pdf.PdfFile* method), 11
`__init__()` (*bp_text.pdf.PdfPage* method), 12
`__init__()` (*bp_text.txt.TxtFile* method), 13

A

`auto_extract` (*bp_text.pdf.PdfFile* property), 11

B

`BibTexDatabase` (class in *bp_text.database*), 6
`bp_text.database` module, 6
`bp_text.language` module, 9
`bp_text.page` module, 8
`bp_text.pdf` module, 10
`bp_text.txt` module, 13
`bp_text.utilities` module, 5

C

`convert_latex_umlauts()` (in module *bp_text.database*), 8
`count_words()` (*bp_text.page.Page* method), 8
`count_words()` (*bp_text.pdf.PdfPage* method), 12

D

`data` (*bp_text.database.BibTexDatabase* property), 7
`data` (*bp_text.page.Page* property), 8
`data` (*bp_text.pdf.PdfFile* property), 11
`data` (*bp_text.pdf.PdfPage* property), 12
`data` (*bp_text.txt.TxtFile* property), 13
`Database` (class in *bp_text.database*), 7

`default_languages` (in module *bp_text.language*), 10
`detect_lang()` (*bp_text.page.Page* method), 8
`detect_lang()` (*bp_text.pdf.PdfPage* method), 12
`detector` (*bp_text.language.LanguageDetector* property), 9

E

`entries` (*bp_text.database.BibTexDatabase* property), 7
`extract_text()` (*bp_text.pdf.PdfFile* method), 11
`extract_text()` (*bp_text.pdf.PdfPage* method), 12
`extract_text_with_ocr()` (*bp_text.pdf.PdfFile* method), 11
`extract_text_without_ocr()` (*bp_text.pdf.PdfFile* method), 11

F

`file` (*bp_text.pdf.PdfFile* property), 11
`file` (*bp_text.pdf.PdfPage* property), 12
`file` (*bp_text.txt.TxtFile* property), 13
`file_checksum` (*bp_text.pdf.PdfFile* property), 11
`file_checksum` (*bp_text.txt.TxtFile* property), 13
`file_checksum()` (in module *bp_text.utilities*), 5
`find_entries()` (*bp_text.database.BibTexDatabase* method), 7

G

`get_entry_by_key()` (*bp_text.database.BibTexDatabase* method), 7
`get_nth_entry()` (*bp_text.database.BibTexDatabase* method), 7
`get_page_label()` (*bp_text.pdf.PdfFile* method), 11
`get_primary_lang()` (*bp_text.pdf.PdfFile* method), 11
`get_primary_lang()` (*bp_text.txt.TxtFile* method), 13

L

`lang` (*bp_text.page.Page* property), 9
`lang` (*bp_text.pdf.PdfFile* property), 11
`lang` (*bp_text.pdf.PdfPage* property), 12
`lang` (*bp_text.txt.TxtFile* property), 13
`LanguageDetector` (class in *bp_text.language*), 9
`languages` (*bp_text.language.LanguageDetector* property), 9

`load()` (*bp_text.database.BibTexDatabase method*), 7
`load()` (*bp_text.database.Database method*), 7

M

module

- `bp_text.database`, 6
- `bp_text.language`, 9
- `bp_text.page`, 8
- `bp_text.pdf`, 10
- `bp_text.txt`, 13
- `bp_text.utilities`, 5

P

`Page` (*class in bp_text.page*), 8
`page_label` (*bp_text.page.Page property*), 9
`page_label` (*bp_text.pdf.PdfPage property*), 12
`page_num` (*bp_text.page.Page property*), 9
`page_num` (*bp_text.pdf.PdfPage property*), 12
`PdfFile` (*class in bp_text.pdf*), 10
`PdfPage` (*class in bp_text.pdf*), 11

R

`read_pickle()` (*in module bp_text.utilities*), 5
`reader` (*bp_text.pdf.PdfFile property*), 11

S

`split_fields_by()` (*bp_text.database.BibTexDatabase method*), 7

T

`text` (*bp_text.page.Page property*), 9
`text` (*bp_text.pdf.PdfPage property*), 12
`TxtFile` (*class in bp_text.txt*), 13

U

`update()` (*bp_text.language.LanguageDetector method*), 9
`update()` (*bp_text.page.Page method*), 9
`update()` (*bp_text.pdf.PdfFile method*), 11
`update()` (*bp_text.pdf.PdfPage method*), 12
`update()` (*bp_text.txt.TxtFile method*), 13

W

`write_pickle()` (*in module bp_text.utilities*), 6